

Lab Report: Week 7 Session B

Capstone Core Development

Mahmudul Hasan (Student ID: 4125999049)

May 22, 2026

Abstract

This report documents Week 7 Session B: implementation of the **Smart Water Integrated Dashboard** capstone. Core features include CSV rainfall loading with threshold alerts, SCS-CN runoff with $Q \leq P$ validation, seven-day reservoir dispatch (total revenue **\$708,849.26**, validation PASS), and DEM-based flood inundation maps. `pytest -q` reports **2 passed**; Streamlit runs at `localhost:8501`. Code pushed to <https://github.com/mahmud456alhasan-debug/smart-water-capstone> (commit 408f211). Evidence: Figure ?? and appendix `prompt_log.md`. OpenCode was not used.

1 Introduction

Session B follows Session A planning (`SCOPE.md`, `ARCHITECTURE.md`) by replacing scaffold stubs with working modules under `src/` and a four-tab Streamlit UI in `app/main.py`.

2 Objectives

- Implement data loading, algorithms, and UI (Exercise 1).
- Document AI interactions in `prompt_log.md` (Exercise 2).
- Self-review tests and physical checks (Exercise 3).

3 Exercise 1: Core features

3.1 Step 1: Data loading

`src/weather/load.py` loads `data/sample_rainfall.csv` with columns `datetime`, `rainfall_mm`. Validation raises if columns are missing.

3.2 Step 2: Core algorithms

Table 1: Implemented modules (reused from Weeks 3–6).

Module	Function
<code>src/weather/alerts.py</code>	GREEN / AMBER / RED vs. mm/h thresholds
<code>src/runoff/scs_cn.py</code>	SCS-CN runoff; enforce $Q \leq P$
<code>src/reservoir/optimizer.py</code>	<code>scipy.optimize.minimize_scalar</code> horizon
<code>src/flood/inundation.py</code>	Wet mask, depth, <code>data/dem.npy</code>

Unit tests: `tests/test_runoff.py`, `tests/test_flood.py`. Fixed `ModuleNotFoundError` via `pytest.ini` (`pythonpath = .`) and `conftest.py`.

Terminal: `pytest -q` → 2 passed in 0.10s.

3.3 Step 3: Streamlit UI

`streamlit run app/main.py` exposes four tabs: Weather & Alerts, Runoff (SCS-CN), Reservoir, Flood map. Figure ?? shows the Weather tab: peak 22 mm/h triggers a **RED** alert with default red threshold 20 mm/h.

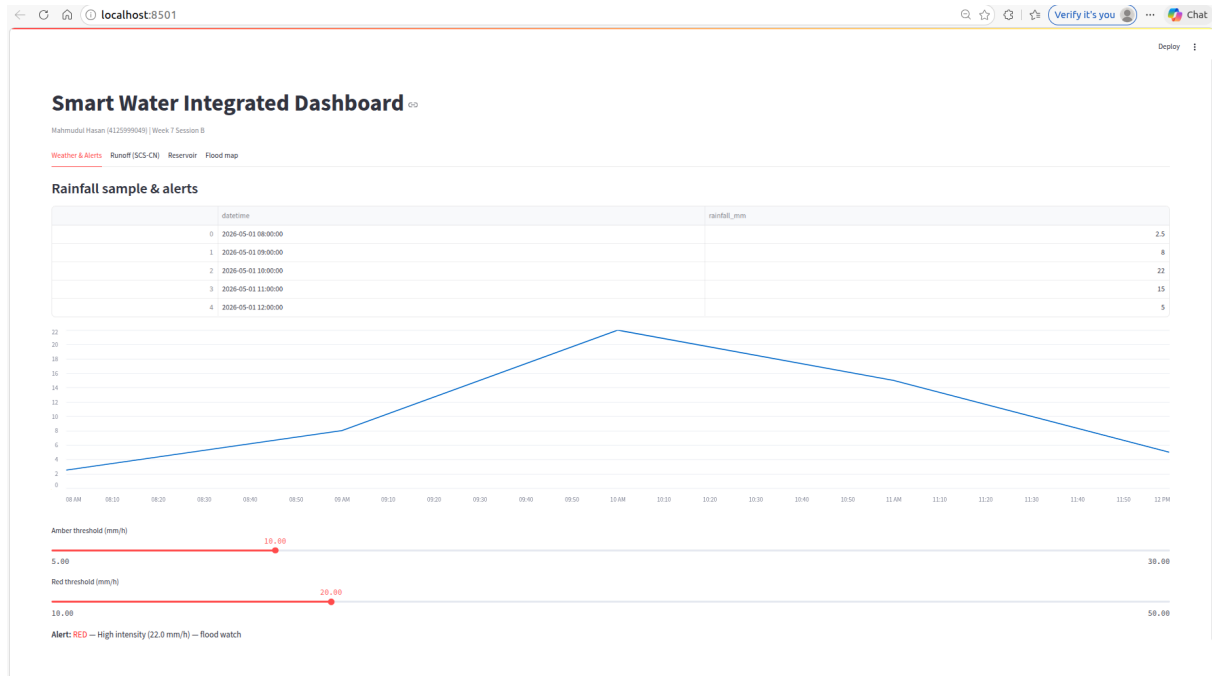


Figure 1: Streamlit dashboard (Weather & Alerts tab) at <http://localhost:8501>.

4 Exercise 2: Prompt log

Session B entries in `prompt_log.md` record the pytest path fix, module wiring prompts, run commands, GitHub push, and screenshot filename. Full file in Appendix ??.

5 Exercise 3: Code review

- Runoff satisfies $Q \leq P$ for tested storms.
- Reservoir schedule matches Week 6 Lab 3 revenue and PASS validation.
- Flood uses `elevation <= flood_level`; DEM bundled in repo.
- No API keys or tokens committed.

6 Exercise 4: Repository update

URL: <https://github.com/mahmud456alhasan-debug/smart-water-capstone>

Commit: Week 7 Session B: Streamlit dashboard and pytest fix (408f211)

Session A planning commits remain on `main`; Session B adds `app/main.py`, `src/` implementations, `data/dem.npy`, and test configuration.

7 Discussion

1. Reusing lab code in `src/` packages kept the one-hour lab feasible.

2. `pythonpath` configuration is required for both `pytest` and Streamlit imports.
3. One dashboard screenshot plus passing tests meets optional submission evidence.

8 Lessons learned

Configure `pytest.ini` when using a `src/` layout. Restart Streamlit after replacing stub tabs. Push to GitHub after each lab session; document commits in `prompt_log.md`.

9 Conclusion

Week 7 Session B objectives were met: working capstone dashboard, unit tests passing, code on GitHub, and documented AI collaboration. Next: Week 8 testing/validation and presentation.

A Streamlit entry: `app/main.py`

`app/main.py` (full file)

```
"""Smart Water Capstone | Streamlit dashboard (Week 7 Session B)."""

from __future__ import annotations

import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.flood.inundation import flood_statistics, load_dem, simulate_flood
from src.reservoir.wrapper import run_baseline_schedule
from src.runoff.scs_cn import scs_runoff_mm
from src.weather.alerts import alert_level
from src.weather.load import load_rainfall_csv

DATA_CSV = ROOT / "data" / "sample_rainfall.csv"
DEM_PATH = ROOT / "data" / "dem.npy"

st.set_page_config(page_title="Smart Water Capstone", layout="wide")
st.title("Smart Water Integrated Dashboard")
st.caption("Mahmudul Hasan (4125999049) | Week 7 Session B")

tab1, tab2, tab3, tab4 = st.tabs(
    ["Weather & Alerts", "Runoff (SCS-CN)", "Reservoir", "Flood map"]
)

with tab1:
    st.subheader("Rainfall sample & alerts")
    df = load_rainfall_csv(str(DATA_CSV))
    st.dataframe(df, use_container_width=True)
    max_mm = float(df["rainfall_mm"].max())
    st.line_chart(df.set_index("datetime")["rainfall_mm"])
    amber = st.slider("Amber threshold (mm/h)", 5.0, 30.0, 10.0, 0.5)
    red = st.slider("Red threshold (mm/h)", 10.0, 50.0, 20.0, 0.5)
    result = alert_level(max_mm, amber=amber, red=red)
```

```

colors = {"GREEN": "green", "AMBER": "orange", "RED": "red"}
st.markdown(
    f"Alert: {colors.get(result.level, 'blue')}{result.level}"
    ↪ {result.message}"
)

with tab2:
    st.subheader("SCS-CN runoff")
    p_mm = st.number_input("Storm rainfall depth P (mm)", 0.0, 200.0, 50.0, 1.0)
    cn = st.number_input("Curve number CN", 1.0, 100.0, 80.0, 1.0)
    try:
        q_mm = scs_runoff_mm(p_mm, cn)
        st.metric("Runoff Q (mm)", f"{q_mm:.2f}")
        st.metric("Runoff / rainfall", f"{(q_mm / p_mm * 100) if p_mm else 0:.1f} %")
        if q_mm > p_mm + 1e-6:
            st.error("Invalid: runoff exceeds rainfall")
        else:
            st.success("Physical check: Q <= P")
        cn_sweep = np.arange(60, 96, 5)
        qs = [scs_runoff_mm(p_mm, c) for c in cn_sweep]
        fig, ax = plt.subplots(figsize=(6, 3.5))
        ax.plot(cn_sweep, qs, "o-")
        ax.set_xlabel("Curve number CN")
        ax.set_ylabel("Runoff Q (mm)")
        ax.set_title(f"SCS-CN sensitivity at P={p_mm:.0f} mm")
        ax.grid(True, alpha=0.3)
        st.pyplot(fig)
        plt.close(fig)
    except (ValueError, TypeError) as exc:
        st.error(str(exc))

with tab3:
    st.subheader("7-day reservoir dispatch")
    eco = st.slider("Eco minimum release (m3/s)", 5.0, 15.0, 10.0, 0.5)
    out = run_baseline_schedule(eco_flow_m3s=eco)
    st.metric("Total revenue (USD)", f"${out['total_revenue_usd']:.2f}")
    st.metric("Validation", out["validation"])
    st.metric("Min storage slack (MCM)", f"{out['storage_slack_mcm']:.3f}")
    st.dataframe(pd.DataFrame(out["schedule"]), use_container_width=True)

with tab4:
    st.subheader("Flood inundation map")
    dem = load_dem(str(DEM_PATH))
    elev_min, elev_max = float(dem.min()), float(dem.max())
    level = st.slider(
        "Flood level (m, absolute stage)",
        elev_min,
        elev_max,
        50.0,
        0.5,
    )
    mask, depth = simulate_flood(dem, level)
    stats = flood_statistics(dem, mask, depth, level)
    c1, c2, c3 = st.columns(3)
    c1.metric("Flooded %", f"{stats.flooded_percent:.2f}")
    c2.metric("Mean depth on wet (m)", f"{stats.mean_depth_m:.2f}")
    c3.metric("Wet cells", stats.wet_cell_count)
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.imshow(dem, cmap="gray", origin="lower")
    depth_ma = np.ma.masked_where(~mask, depth)
    im = ax.imshow(depth_ma, cmap="Blues", origin="lower", alpha=0.65, vmin=0)
    ax.set_title(f"Flood level {level:.1f} m")
    fig.colorbar(im, ax=ax, label="Depth (m)")

```

```
st.pyplot(fig)
plt.close(fig)
```

B Runoff module: src/runoff/scs_cn.py

runoff_scs_cn.py (excerpt file)

```
"""SCS Curve Number runoff (from week5_session_b_lab2)."""

from __future__ import annotations

from typing import Union

_RETENTION_FACTOR = 25400.0
_RETENTION_OFFSET = 254.0
_INITIAL_ABSTRACTION_RATIO = 0.2
_MIN_CN = 1
_MAX_CN = 100

def _validate_positive_rainfall(p_mm: Union[int, float]) -> float:
    if not isinstance(p_mm, (int, float)):
        raise TypeError(f"P must be a number, got {type(p_mm).__name__}")
    p_mm = float(p_mm)
    if p_mm < 0:
        raise ValueError(f"Rainfall depth P cannot be negative, got {p_mm}")
    return p_mm

def _validate_curve_number(cn: Union[int, float]) -> float:
    if not isinstance(cn, (int, float)):
        raise TypeError(f"CN must be a number, got {type(cn).__name__}")
    cn = float(cn)
    if not (_MIN_CN <= cn <= _MAX_CN):
        raise ValueError(f"Curve number CN must be in [{_MIN_CN}, {_MAX_CN}], got {cn}")
    return cn

def scs_runoff_mm(p_mm: Union[int, float], cn: float) -> float:
    """Direct runoff Q (mm); enforced 0 <= Q <= P."""
    p_mm = _validate_positive_rainfall(p_mm)
    cn = _validate_curve_number(cn)
    s_mm = _RETENTION_FACTOR / cn - _RETENTION_OFFSET
    ia_mm = _INITIAL_ABSTRACTION_RATIO * s_mm
    if p_mm <= ia_mm:
        return 0.0
    effective = p_mm - ia_mm
    q_mm = effective**2 / (effective + s_mm)
    return max(0.0, min(q_mm, p_mm))
```

C Submitted prompt log: prompt_log.md

prompt_log.md (full file, Sessions A + B)

```
# Prompt Log - Week 7 Session A (Capstone Planning)

**Student:** Mahmudul Hasan (4125999049)
**Tool:** Cursor Agent (Auto); OpenCode not used
```

```

---

## Exercise 1 - Project scope

**AI used?** Partial scope drafted from completed Weeks 3--6 lab work, then saved to
↳ `SCOPE.md`.

**Prompt (summary):** Integrate week5_session_a_lab1 (weather/alerts),
↳ week5_session_b_lab2 (SCS-CN), week6_session_a_lab3 (reservoir), week6_session_b_lab4
↳ (flood) into one Streamlit capstone; list functional/non-functional requirements and
↳ scope boundaries.

**Output file:** `SCOPE.md`

**Changes made:** Fixed title "Smart Water Integrated Dashboard"; excluded production
↳ deployment and real GIS DEM from scope.

---

## Exercise 2 - Architecture design

**Date:** 2026-05-21

**Prompt (from lab guide, filled):**

```text
I want to build a Smart Water Integrated Monitoring and Decision Support Dashboard.

Requirements:
1. Load or fetch rainfall/weather data (CSV default; optional API).
2. Compute SCS-CN runoff with physical checks (runoff <= rainfall).
3. Run 7-day reservoir release optimization (scipy; MCM storage; m3/s flows).
4. Flood inundation from 100x100 synthetic DEM and flood_level (m).
5. Streamlit UI with tabs: Weather/Alerts, Runoff, Reservoir, Flood map.
6. prompt_log.md and AGENTS.md for AI-assisted development.

Please help me design:
1. System architecture diagram (describe components)
2. Directory structure
3. Main modules and their responsibilities
4. Data flow between components
5. Technology stack recommendations
6. API design (if applicable)
```

**Summary:** Five-layer flow (weather runoff reservoir flood Streamlit). Modular
↳ `src/` packages mapped to prior labs. Internal Python function API; CSV-first for
↳ reproducibility.

**Output file:** `ARCHITECTURE.md`

**Changes made:** ASCII diagram for LaTeX/report compatibility; no FastAPI (unlike week2
↳ demo) to keep capstone scope teachable.

---

## Exercise 3 - Technical plan / scaffold

**Prompt (from lab guide):**

```text
Based on our architecture design, please generate:
1. Complete directory structure

```

```

2. Main file templates with docstrings
3. requirements.txt with all dependencies
4. README.md with setup instructions
5. Initial AGENTS.md template
6. Sample data files (if needed)

Make it ready for development in Week 7 Session B. Python 3.8+, Streamlit, numpy, pandas,
↳ matplotlib, scipy.
```

**Summary:** Created `app/main.py` stub, `src/{weather,runoff,reservoir,flood}` packages,
↳ `tests/`, `data/sample_rainfall.csv`, README, AGENTS.md, requirements.txt.

**Changes made:** Stubs only | implementation deferred to Session B.

---

## Exercise 4 - Repository

**GitHub URL:** https://github.com/mahmud456alhasan-debug/smart-water-capstone

**Planned repo name:** `smart-water-capstone`

**Commit message:** `Week 7 Session A: capstone scope and architecture`

**Push result:** `main` branch, 30 objects, tracking `origin/main`.

**Report screenshots:** `week7_session_a_github_page_a.png` (file tree),
↳ `week7_session_a_github_page_b.png` (README).

---

## Lessons learned

Planning works best when capstone scope explicitly reuses lab modules instead of starting
↳ a new stack. Documenting wet-cell and unit conventions early avoids Week 8 debugging.
↳ Cursor can generate scaffold and report artifacts in one session; GitHub push and one
↳ screenshot remain manual. Cross-checking architecture with a second LLM
↳ (ChatGPT/Gemini) is optional but useful before Session B coding.

---

# Week 7 Session B | Core Development

**Tool:** Cursor Agent | **Date:** 2026-05-22

## Feature: pytest path + modules

**Issue:** `ModuleNotFoundError: No module named 'src'` when running `pytest -q`.

**Fix:** Added `pytest.ini` (`pythonpath = .`) and `conftest.py` to insert project root
↳ on `sys.path`. Same pattern in `app/main.py` for Streamlit.

**Result:** `2 passed in 0.11s`

## Feature: Streamlit four-tab dashboard

**Prompt (summary):** Wire Week 56 lab logic into capstone `src/` packages and replace
↳ `app/main.py` stubs with Weather, Runoff, Reservoir, and Flood tabs.

**Implementation:**
- `src/runoff/scs_cn.py` | full SCS-CN with  $Q \leq P$ 
- `src/reservoir/optimizer.py` + `wrapper.py` | 7-day schedule (~$708,849)

```

```
- `src/flood/inundation.py` | DEM load + map
- `src/weather/` | CSV load + GREEN/AMBER/RED alerts
- `data/dem.npy` copied from week6_session_b_lab4

**Run:** `streamlit run app/main.py` → http://localhost:8501

**GitHub push:** commit `408f211` on `main`
↳ https://github.com/mahmud456alhasan-debug/smart-water-capstone

**Screenshot:** `week7_session_b_Streamlit_page.png` (Weather & Alerts tab, RED alert at
↳ 22 mm/h)

## Lessons learned (Session B)

Add `pytest.ini` early when using a `src/` layout. Restart Streamlit after code changes.
↳ One screenshot of the dashboard plus `pytest -q` terminal output is enough for
↳ submission.
```